

Dining Philosophers Problem

Synchronization Problem and Semaphore-Based Solutions

Prashant Jadav

Department of Computer Engineering
DEPSTAR – CHARUSAT

Dining Philosophers Problem

Problem

Five philosophers sit around a circular table. Between every two philosophers there is one fork. Each philosopher alternates between:

- **Thinking**
- **Hungry**
- **Eating**

Thinking → Hungry → Eating → Thinking

Resource requirement

To eat, a philosopher needs **both** the left and right forks. A fork can be used by only one philosopher at a time.

Dining Philosophers: Resource Arrangement

Philosopher	Left Fork	Right Fork
P_0	F_0	F_1
P_1	F_1	F_2
P_2	F_2	F_3
P_3	F_3	F_4
P_4	F_4	F_0

Circular dependency

P_i needs F_i and $F_{(i+1) \bmod 5}$

Synchronization challenge

Neighboring philosophers share a fork, so adjacent philosophers cannot eat simultaneously.

Naive Semaphore Solution

Assume one binary semaphore for each fork.

$$\text{fork}[0..4] = 1$$

Each fork is represented by a binary semaphore initialized to 1.

Philosopher algorithm:

```
wait(left_fork);  
wait(right_fork);
```

```
EAT();
```

```
signal(right_fork);  
signal(left_fork);
```

```
THINK();
```

Why does this look correct?

A philosopher holds both forks before eating, so two neighboring philosophers cannot use the same fork simultaneously.

How Deadlock Occurs

Suppose all philosophers become hungry simultaneously.

Philosopher	Current state
P_0	Holds F_0 , waits for F_1
P_1	Holds F_1 , waits for F_2
P_2	Holds F_2 , waits for F_3
P_3	Holds F_3 , waits for F_4
P_4	Holds F_4 , waits for F_0

$$P_0 \rightarrow F_1 \rightarrow P_1 \rightarrow F_2 \rightarrow P_2 \rightarrow F_3 \rightarrow P_3 \rightarrow F_4 \rightarrow P_4 \rightarrow F_0$$

Result

A circular wait is created. No philosopher can obtain the second fork. Hence, the system is in **deadlock**.

Why the Naive Solution Can Deadlock

The deadlock satisfies the four Coffman conditions:

- ① **Mutual exclusion:** A fork is held by only one philosopher.
- ② **Hold and wait:** A philosopher holds one fork while waiting for another.
- ③ **No preemption:** A fork cannot be forcibly taken away.
- ④ **Circular wait:** Each philosopher waits for a fork held by another philosopher.

Break any one condition $\implies$ prevent deadlock

Solution 1: Allow at Most Four Philosophers

Use a counting semaphore: `room=4`

```
wait(room);
```

```
wait(left_fork);
```

```
wait(right_fork);
```

```
EAT();
```

```
signal(right_fork);
```

```
signal(left_fork);
```

```
signal(room);
```

```
THINK();
```

Why does it work?

Only four philosophers can compete for five forks. Therefore, all five philosophers cannot simultaneously hold one fork, so a complete circular wait cannot form.

Solution 2: Asymmetric Fork Ordering

Rule

- P_0-P_3 : acquire the left fork first.
- P_4 : acquire the right fork first.

Philosophers P_0-P_3

```
wait(left_fork);  
wait(right_fork);
```

```
EAT();
```

```
signal(right_fork);  
signal(left_fork);
```

Philosopher P_4

```
wait(right_fork);  
wait(left_fork);
```

```
EAT();
```

```
signal(left_fork);  
signal(right_fork);
```

Key idea

Different fork acquisition order breaks the circular-wait condition.

Solution 3: State-Based Semaphore Solution

Use philosopher states and a private semaphore for each philosopher.

```
enum { THINKING, HUNGRY, EATING };

semaphore mutex = 1;
semaphore self[5] = {0, 0, 0, 0, 0};

take_forks(i):
    wait(mutex);
    state[i] = HUNGRY;
    test(i);
    signal(mutex);
    wait(self[i]);

put_forks(i):
    wait(mutex);
    state[i] = THINKING;
    test(LEFT(i));
    test(RIGHT(i));
    signal(mutex);
```

A hungry philosopher is allowed to eat only when both neighbors are not eating.

State-Based Solution: test(i)

```
test(i):  
    if state[i] == HUNGRY  
        and state[LEFT(i)] != EATING  
        and state[RIGHT(i)] != EATING  
    then  
        state[i] = EATING;  
        signal(self[i]);
```

Interpretation

- The left neighbor must not be eating.
- The right neighbor must not be eating.
- If both conditions hold, philosopher i can eat.

HUNGRY $\rightarrow$ test $\rightarrow$ EATING

Deadlock Prevention vs Starvation

Solution	Deadlock	Main concern
Naive solution	Possible	Deadlock
At most four philosophers	Prevented	Reduced concurrency
Asymmetric ordering	Prevented	Fairness / starvation
State-based solution	Prevented	Fairness / starvation

Important distinction

Deadlock: affected processes cannot proceed.

Starvation: one philosopher may wait indefinitely while others continue to make progress.

Dining Philosophers: Key Takeaways

- 1 Each philosopher needs two shared resources to eat.
- 2 The naive “left fork then right fork” solution can deadlock.
- 3 Deadlock occurs because all four Coffman conditions can hold.
- 4 Deadlock can be prevented by:
 - limiting the number of competing philosophers,
 - changing resource acquisition order, or
 - coordinating philosopher states.
- 5 Preventing deadlock does not automatically guarantee fairness.
- 6 **Starvation** must be considered separately from deadlock.

Exam Perspective

Be able to identify the deadlock sequence and explain which deadlock condition is broken by each solution.